

Departamento de Ciencias de la Computación

Asignatura:

Modelos Discretos para la Ingeniería

Tema:

Análisis Matemático y Estructural del Sistema de Evaluación Académica

Asesor:

Mgtr. Washington Eduardo Loza Herrera

Integrantes:

Chavez Oscullo Lizbeth Liliana

Galarza Alvaro Matías Miguel

Iza Caiza Melany Guadalupe

Jácome Calahorrano Micaela Victoria

NRC: 30627

21/06/2026

Contenido

Introducción	3
Objetivo	3
Objetivo general	3
Objetivos específicos	3
Modelo lógico del sistema	4
Principios de conteo	4
Combinaciones	5
Probabilidad discreta y transiciones	6
Demostración por casos	7
Inducción sobre las condiciones	7
Inducción sobre los estudiantes	7
Conjunto parcialmente ordenado	7
Diagrama de Hasse	8
Complejidad computacional	9
Motor de inferencia	9
Evaluación de n estudiantes	9
Complejidad de las operaciones estructurales	10
Implementación estructural	11
Memoria dinámica	11

Prueba funcional	12
Conclusiones	13
Referencias	14

Introducción

El código empleado a manera de Sistema de Evaluación Académica clasifica a los estudiantes a partir de tres condiciones: promedio, asistencia y cumplimiento de actividades. Estas variables se expresan mediante proposiciones booleanas y producen tres estados posibles: reprobado, recuperación y aprobado.

La ampliación desarrollada incorpora herramientas de matemáticas discretas para explicar cómo se generan, organizan y procesan dichos estados.

El análisis aplica conteo, combinaciones, probabilidad, inducción, relaciones de orden, Diagrama de Hasse y complejidad a funciones reales del programa en C++, organizado mediante Modelo-Vista-Controlador.

Objetivo

Objetivo general

Analizar matemática y estructuralmente el Sistema de Evaluación Académica e integrar funciones de conteo, probabilidad, demostración, orden parcial, Hasse y complejidad.

Objetivos específicos

- Determinar las configuraciones posibles y clasificarlas mediante las reglas lógicas.
- Calcular combinaciones y probabilidades de estados y transiciones.
- Demostrar la corrección de la clasificación para cualquier cantidad finita de estudiantes.
- Representar los estados mediante un orden parcial y su Diagrama de Hasse.
- Calcular la complejidad temporal y espacial de las principales operaciones implementadas.

Desarrollo

Modelo lógico del sistema

El programa define P como “promedio mayor o igual a 7”, Q como “asistencia mayor o igual al 80 %” y R como “actividades completas”. Cada proposición tiene un valor verdadero o falso. Las reglas de inferencia son: aprobado cuando $P \wedge Q \wedge R$; recuperación cuando $(\neg P \wedge Q) \vee (P \wedge Q \wedge \neg R)$; y reprobado en cualquier otro caso.

```
EstadoAcademico deducirEstado(bool p, bool q, bool r) {  
    if (p && q && r) return APROBADO;  
    if ((!p && q) || (p && q && !r)) return RECUPERACION;  
    return REPROBADO;  
}
```

Principios de conteo

El principio multiplicativo indica que, cuando un proceso se divide en etapas independientes, la cantidad total de resultados se obtiene multiplicando las opciones de cada etapa (Kirk, 2023). Como P, Q y R poseen dos valores, el espacio lógico contiene $2 \times 2 \times 2 = 2^3 = 8$ configuraciones.

Tabla 1

Configuraciones lógicas y estado académico

P	Q	R	Estado
F	F	F	Reprobado
F	F	V	Reprobado
F	V	F	Recuperación
F	V	V	Recuperación
V	F	F	Reprobado
V	F	V	Reprobado
V	V	F	Recuperación
V	V	V	Aprobado

Resultado: cuatro configuraciones producen reprobación, tres recuperaciones y una aprobación.

Combinaciones

Las combinaciones se utilizan cuando interesa seleccionar elementos sin considerar el orden. En el sistema, alcanzar primero el promedio y después la asistencia produce el mismo subconjunto {P, Q} que cumplirlos en el orden contrario. La expresión general es:

$$C(n, r) = \frac{n!}{r!(n-r)!}$$

Tabla 2

Combinaciones de los tres requisitos académicos

r	Resultado	Subconjunto	Interpretación
0	$C(3,0)=1$	{ }	Ningún requisito
1	$C(3,1)=3$	{P}, {Q}, {R}	Un requisito
2	$C(3,2)=3$	{P,Q}, {P,R}, {Q,R}	Dos requisitos
3	$C(3,3)=1$	{P,Q,R}	Tres requisitos

$$C(3, 0) + C(3, 1) + C(3, 2) + C(3, 3) = 1 + 3 + 3 + 1 = 8$$

La igualdad coincide con las ocho configuraciones binarias. Sin embargo, dos subconjuntos de igual tamaño no siempre producen el mismo estado. Por ejemplo, {P,R} conduce a reprobación porque no se cumple Q, mientras que {P,Q} conduce a recuperación porque falta R. Por tanto, el sistema debe identificar cuáles requisitos se cumplen, no solo cuántos.

```
for (int r = 0; r <= 3; r++) {  
    int resultado = combinacion(3, r);  
    suma += resultado;  
}
```

La función calcula los cuatro coeficientes binomiales. Como $n=3$ es fijo, el costo dentro de esta aplicación es constante; en una generalización, el costo depende del procedimiento utilizado para calcular factoriales.

Probabilidad discreta y transiciones

Cuando los resultados de un espacio muestral son equiprobables, la probabilidad teórica de un evento es la razón entre sus casos favorables y el total de resultados (Kirk, 2023). Bajo este supuesto, las ocho configuraciones generan la siguiente distribución:

Tabla 3

Distribución teórica de los estados académicos

Estado	Fracción	Decimal	Porcentaje
Reprobado	4-ago	0.5000	50.00 %
Recuperación	3-ago	0.3750	37.50 %
Aprobado	1-ago	0.1250	12.50 %

Las transiciones se calculan con el historial real del programa. Si N_{ij} es el número de estudiantes que pasó del estado i al estado j y N_i es el total que inició en i , entonces $P(j|i) = N_{ij} / N_i$. Esta es una probabilidad empírica porque utiliza resultados observados (Kirk, 2023).

$$P(\text{Aprobado} | \text{Recuperación}) = N(\text{Recuperación} \rightarrow \text{Aprobado}) / N(\text{Recuperación})$$

```
double HistorialTransiciones::probabilidad(  
    EstadoAcademico origen, EstadoAcademico destino) const {  
    int totalOrigen = contarOrigen(origen);  
    if (totalOrigen == 0) return 0.0;  
    return static_cast<double>(contarTransicion(origen, destino))  
        / totalOrigen;  
}
```

En la prueba funcional se registraron dos estudiantes en recuperación: uno pasó a aprobación y otro permaneció en recuperación. Por ello,

$$P(\text{Aprobado} | \text{Recuperación}) = 1/2 = 0.5000$$

$$P(\text{Recuperación} | \text{Recuperación}) = 1/2 = 0.5000$$

Demostración del funcionamiento

Demostración por casos

La tabla de verdad contiene las 2^3 configuraciones y verifica todo el dominio. Aprobación exige P, Q y R verdaderas; recuperación exige $\neg P$ o $\neg R$ bajo las condiciones indicadas; y la sentencia final recibe los casos restantes. Por ello, las reglas son completas, excluyentes y asignan un único estado.

Inducción sobre las condiciones

Proposición: m condiciones binarias generan 2^m configuraciones. Para $m=1$ existen $2=2^1$ casos. Si k condiciones producen 2^k casos, una condición nueva duplica cada configuración; entonces $2 \cdot 2^k = 2^{k+1}$. Por inducción, la propiedad se cumple para todo $m \geq 1$ (Epp, 2020).

Inducción sobre los estudiantes

Proposición: el motor clasifica correctamente a “n” estudiantes. Para $n=1$, P, Q y R producen un estado único. Si los primeros “k” fueron clasificados correctamente, el estudiante $k+1$ se desencola, se evalúa y se agrega a un solo conjunto sin alterar los anteriores. Por inducción, el procedimiento es correcto para todo $n \geq 1$.

La cola aplica FIFO y la pila LIFO; ambas cambian el orden de procesamiento, pero no las reglas de clasificación.

Conjunto parcialmente ordenado

Sea $E = \{\text{Reprobado}, \text{Recuperación}, \text{Aprobado}\}$. Se define $x \leq y$ cuando y representa un nivel académico igual o superior a x. Para que $(E, \leq)$ sea un conjunto parcialmente ordenado, la relación debe ser reflexiva, antisimétrica y transitiva (Rosen, 2019).

Tabla 4*Propiedades de la relación de orden académico*

Propiedad	Condición	Aplicación al sistema
Reflexiva	$x \leq x$ para todo $x \in E$.	Cada estado es comparable consigo mismo.
Antisimétrica	Si $x \leq y$ y $y \leq x$, entonces $x = y$.	Dos niveles diferentes no pueden ser superiores entre sí.
Transitiva	Si $x \leq y$ y $y \leq z$, entonces $x \leq z$.	Reprobado $\leq$ Recuperación y Recuperación $\leq$ Aprobado implican Reprobado $\leq$ Aprobado.

$$\leq = \{(R, R), (C, C), (A, A), (R, C), (C, A), (R, A)\}$$

Todos los pares de estados son comparables, de modo que el orden parcial también es una cadena. El mínimo es Reprobado y el máximo es Aprobado. Las relaciones de cobertura son Recuperación sobre Reprobado y Aprobado sobre Recuperación.

Diagrama de Hasse

El Diagrama de Hasse elimina los lazos reflexivos y las relaciones obtenidas por la transitividad. Por ello, no se dibuja una conexión directa entre Reprobado y Aprobado, ya que esta relación se deduce mediante Recuperación.

Figura 1*Diagrama de Hasse de los estados académicos*

Complejidad computacional

El análisis utiliza operaciones elementales (OE): asignaciones, comparaciones, avances y operaciones lógicas de costo constante. Los coeficientes dependen del modelo adoptado, pero la clasificación asintótica permanece estable. La notación O expresa una cota superior, Ω una cota inferior y Θ una cota ajustada (Cormen et al., 2022).

Motor de inferencia

El motor siempre evalúa tres proposiciones y un número fijo de operadores. En el peor caso, el modelo implementado estima 16 OE:

$$T_{inf}(1) = 16 OE = \Theta(1)$$

Evaluación de n estudiantes

El método ejecutarMotorInferencia limpia los conjuntos actuales, encola “ n ” punteros y procesa cada estudiante una sola vez. Las funciones del programa presentan las siguientes expresiones:

$$T_1(n) = 4n + 3 \text{ (limpieza de conjuntos)}$$

$$T_2(n) = 7n + 2 \text{ (encolado de punteros)}$$

$$T_3(n) = 24n + 1 \text{ (evaluación y clasificación)}$$

$$T(n) = T_1(n) + T_2(n) + T_3(n) = 35n + 6 OE$$

$$O(n), \Omega(n) \text{ y } \Theta(n)$$

La cota inferior también es lineal porque todo estudiante debe ser leído y clasificado al menos una vez. En memoria, el repositorio y los conjuntos almacenan $\Theta(n)$ registros; durante la evaluación, la cola puede contener hasta n referencias.

Complejidad de las operaciones estructurales

Tabla 5

Complejidad temporal y espacial de las operaciones

Operación	Tiempo	Espacio	Justificación
deducirEstado	$\Theta(1)$	$\Theta(1)$	Tres proposiciones fijas.
agregarFinal	$\Theta(1)$	$\Theta(1)$	Usa el puntero ultimo.
agregar con validación	$\Theta(n)$	$\Theta(1)$	Busca nombre duplicado.
n registros con validación	$\Theta(n^2)$	$\Theta(n)$	La búsqueda crece con la lista.
buscar o eliminar	$\Theta(n)$	$\Theta(1)$	Recorrido secuencial.
intersección/diferencia	$O(a \cdot b)$	$\Theta(a)$	Cada elemento busca en la otra lista.
unión de conjuntos	$O((a+b)^2)$	$\Theta(a+b)$	Evita duplicados con búsquedas.
probabilidad de transición	$\Theta(h)$	$\Theta(1)$	Recorre h registros.
8 configuraciones fijas	$\Theta(1)$	$\Theta(1)$	Tres condiciones constantes.
m condiciones	$\Theta(2^m)$	$\Theta(2^m)^*$	Enumera todas las configuraciones.
Hasse de 3 estados	$\Theta(1)$	$\Theta(1)$	Tres vértices y dos aristas.
Hasse de s estados en cadena	$\Theta(s)$	$\Theta(s)$	s vértices y s-1 aristas.

Nota. El espacio es $\Theta(2^m)$ solo si todas las configuraciones se almacenan; si se muestran una por una, el espacio auxiliar puede ser $\Theta(m)$.

La relación con el Diagrama de Hasse es directa: el sistema posee tres niveles fijos, por lo que aumentar la cantidad de estudiantes no crea nuevos vértices; cada estudiante se ubica en uno de esos niveles en tiempo constante y el procesamiento total crece linealmente con n .

Implementación estructural

El proyecto conserva el patrón Modelo-Vista-Controlador. El Modelo contiene las reglas lógicas, listas, pila, cola, historial y relación de orden; la Vista valida las entradas y presenta resultados y finalmente el Controlador coordina la evaluación, recuperación, conteo, probabilidades, Hasse y complejidad.

Tabla 6

Responsabilidades de los componentes MVC

Componente	Responsabilidad
Modelo	Estudiante, Lista, Pila, Cola, HistorialTransiciones, deducirEstado y cubreA.
Vista	Menú, lectura validada, presentación de listas, resultados y resúmenes.
Controlador	Flujo de opciones, clasificación, recuperación, conteo, probabilidades y análisis.

Memoria dinámica

Las listas poseen destructor, constructor de copia, asignación por copia y operaciones de movimiento. Esta implementación evita pérdidas de memoria y permite devolver resultados de

unión, intersección y diferencia por valor. La cola y la pila almacenan punteros a los estudiantes del repositorio, por lo que la actualización de una transición modifica el registro original.

```
void Lista::agregarFinal(const Estudiante& estudiante) {
    Nodo* nuevo = new Nodo{estudiante, nullptr};
    if (cabeza == nullptr) cabeza = ultimo = nuevo;
    else { ultimo->siguiente = nuevo; ultimo = nuevo; }
    cantidad++;
}
```

El puntero ultimo evita recorrer toda la lista para insertar. Esta decisión es esencial para que la clasificación en los conjuntos A, B y C permanezca en $\Theta(n)$.

Prueba funcional

Se ejecutó una prueba con cuatro estudiantes para comprobar la tabla lógica, la reconstrucción de conjuntos y las transiciones de recuperación.

Tabla 7

Datos y clasificación inicial de la prueba

Nombre	Promedio	Asistencia	Tareas	Estado inicial
Ana	8.0	90%	Sí	Aprobado
Bruno	6.0	85%	Sí	Recuperación
Carla	8.0	90%	No	Recuperación
Diego	9.0	70%	Sí	Reprobado

El resumen inicial fue: 1 aprobado, 2 en recuperación y 1 reprobado. Después se procesaron los dos exámenes mediante la pila. Carla obtuvo promedio 8 y completó actividades; Bruno mantuvo promedio 6. El resultado final fue: 2 aprobados, 1 en recuperación y 1 reprobado.

Tabla 8

Probabilidades observadas desde recuperación

Transición	Cálculo	Probabilidad
Recuperación → Reprobado	0/2	0.0000
Recuperación→ Recuperación	1/2	0.5000
Recuperación → Aprobado	1/2	0.5000

Las probabilidades suman 1.0000, lo que confirma que cada transición registrada desde recuperación termina en exactamente uno de los tres estados. Asimismo, la suma de clasificados coincide con el total de estudiantes, por lo que los conjuntos son mutuamente excluyentes y exhaustivos.

La compilación con C++17 se realizó sin errores. Las validaciones rechazaron nombres vacíos, valores fuera de rango y opciones no numéricas, mientras que los destructores liberaron los nodos creados por las estructuras dinámicas.

Conclusiones

El principio multiplicativo determinó ocho configuraciones para P, Q y R. La distribución obtenida fue de cuatro casos de reprobación, tres de recuperación y uno de aprobación. Las combinaciones confirmaron la identidad $1+3+3+1=8$, aunque también evidenciaron que el tipo de requisito cumplido influye en el resultado.

El historial permitió calcular probabilidades condicionales reales entre estados. En la prueba realizada, la mitad de los estudiantes en recuperación pasó a aprobación y la otra mitad permaneció en recuperación. Este cálculo es diferente de la distribución teórica de las ocho configuraciones y depende de los datos observados.

La demostración por casos verificó todas las entradas posibles, mientras que la inducción justificó el número 2^m de configuraciones y la corrección del motor para cualquier cantidad finita de estudiantes. Los estados forman la cadena Reprobado $\leq$ Recuperación $\leq$ Aprobado, representada mediante un Diagrama de Hasse de tres niveles.

La evaluación completa tiene complejidad $\Theta(n)$, debido a que cada estudiante se procesa una vez y las inserciones de clasificación usan un puntero al último nodo. Sin embargo, el registro con búsqueda de duplicados y algunas operaciones de conjuntos pueden alcanzar complejidad cuadrática. El Hasse permanece constante porque aumentar estudiantes no modifica la cantidad de estados.

La separación MVC, el uso de memoria dinámica y la validación de entradas permiten que el análisis matemático se refleje en funciones verificables del programa, manteniendo una relación clara entre teoría de modelos discretos y desarrollo en C++.

Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press. <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>
- Epp, S. S. (2020). Discrete mathematics with applications (5th ed.). Cengage.
- Kirk, D. (2023). Contemporary mathematics. OpenStax. <https://openstax.org/books/contemporary-mathematics/pages/1-introduction>
- Rosen, K. H. (2019). Discrete mathematics and its applications (8th ed.). McGraw Hill.